

Freight Rate Prediction from Tabular Shipment Data

Spotter ML Engineer assessment report

Prosper Arineitwe Asiimwe

Email: prox0591@gmail.com

Github Repository: <https://github.com/ProsperAsiimwe/spotter>

Live Demo of model prediction: <https://prosperasiimwe.dev/freight>

Abstract

This report covers the two items the assessment asks for in writing: how the labeled development file was split and validated, and the fixed December chart produced by the provided `score.py`. The labeled development data is `data/train-test.csv` (48,000 loads, January to October 2025). Every row in `data/validation.csv` (12,000 loads, November to December, no target) is scored into `validation_predictions.csv` as `load_id, predicted_rate`. Because that scoring period lies in the future, model selection uses a January to August / September to October chronological holdout rather than a random split or k -fold. LightGBM attains the lowest holdout MAE of \$108.58, against \$109.90 for XGBoost, \$116.51 for CatBoost, and \$121.01 for Ridge. That configuration is refit on all 48,000 labeled rows and used for the 12k file and the December chart. Distance is the strongest single correlate of posted rate. Missing values, unseen cities, and the absence of market variables from the December series are handled in one feature builder.

1 Introduction

Freight rate prediction is a regression problem because I was asked to estimate the posted price of a truck load from information available at quotation time. Rates vary with features like distance, equipment, weight, origin and destination, market conditions, and calendar effects. A useful model has to capture nonlinear interactions among those variables without leaking information from later shipments into training.

The assessment names `data/train_test.csv` as labeled development data; the file on the github repository is `data/train-test.csv` (48,000 shipments, January through October 2025). `data/validation.csv` holds the 12,000 loads that require final predictions, with no `posted_rate`. I

chose to look at this modelling problem as a classical temporal generalization pattern (Madaan et al., 2025). The analysis treats the data as tabular regression rather than as a graph or a per-lane time series. That choice follows from lane sparsity: there are 10,793 unique combinations of pickup city, delivery city, and equipment, with a median of three observations per combination. Lane-specific history is noisy under that density, and more elaborate spatial-temporal models have little repeated structure to use.

The rest of the report covers three design decisions. First, a chronological split that matches the forecasting setting. Second, a comparison of three gradient-boosted tree families, including LightGBM (Ke et al., 2017), XGBoost (Chen and Guestrin, 2016), and CatBoost (Prokhorenkova et al., 2018), against simple baselines, with all models trained and evaluated on the same chronological split. Third, how the fitted model behaves when market variables are missing from the December series.

2 Data and Problem Formulation

The labeled development data has 48,000 observations, with IDs ranging from TR-000001 through TR-048000, each with a `posted_rate`, 1 January through 31 October 2025.

Validation data to score has 12,000 observations, with IDs ranging from TE-000001 through TE-012000, November and December 2025. The target is withheld. `data/validation-predictions-template.csv` lists the same 12,000 `load_id` values. The submitted file `validation_predictions.csv` fills `predicted_rate` for every one of those IDs.

A separate 31-day series (`./december-chart-inputs.csv`) is used only for the December chart. Every row is the same Lexington to Fort Wayne lane: 360 miles, Dry Van, 32,000 lb. Latitude, lon-

gitude, market_index, and quote_signal are not provided.

The target is modelled on a log scale,

$$z_i = \log(1 + y_i), \quad (1)$$

and inverted with $\hat{y}_i = \exp(\hat{z}_i) - 1$. This keeps the training loss closer to dollars without letting a few very large loads dominate a squared-error fit.

3 Train / test split and validation

The assessment asks how the development data was split and validated. The labeled file is cut on calendar date, with no shuffling.

The training period is 1 January through 31 August 2025 (38,477 shipments). The holdout is 1 September through 31 October 2025 (9,523 shipments). That holdout is used only to compare models. It is the last labeled window before the November-December file we actually have to score.

A random split would place September and October loads on both sides of the cut and make holdout MAE look better than a true future-period test. Imputers, city lookups, and equipment medians are fit on January-August only during model selection. After the family and hyperparameters are locked, the same configuration is refit on all 48,000 labeled rows before every row of validation.csv is scored.

The random seed for training is 42 throughout.

4 Exploratory Analysis

Distance is the dominant individual predictor. Its Pearson correlation with posted_rate is 0.909. The median posted rate is about \$2,031 on a median haul of 953 miles. Equipment still moves dollars per mile: the medians are \$2.31 for Reefer, \$2.22 for Flatbed, and \$2.05 for Dry Van (Figure 2).

Weekend effects are negligible. Median posted rate is \$2,030 on weekdays and \$2,032 on weekends. The time pattern that matters is month: dollars per mile peak in June and are lower in January (Figure 1).

quote_signal is not a ready-made rate. Its correlation with dollars per mile is 0.05, and multiplying it by distance does not beat distance alone.

The labeled file has 64 distinct cities; validation has 72. Eight cities appear only in validation: Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, and San Diego. Any city encoding has to survive unseen names.

Column	Labeled (48k)	Validation (12k)
weight	300	165
market_index	374	249

Table 1: Missing values in selected numerical columns.

Lane-level sparsity further limits the modelling options. There are 10,793 pickup-delivery-equipment combinations, with a median of three observations each. That is too thin for reliable lane history. The December chart is a stronger case of the same point: origin, destination, equipment, and miles are fixed for 31 days, so lane history would be a constant and could not produce a curve.

The Lexington-Fort Wayne Dry Van lane has 21 labeled rows and a median posted rate of \$808. Chart distance is 360 miles, against 351-379 miles on those labeled rows.

5 Data Quality and Missing Values

Missing values appear in both the labeled file and the validation file, so dropping rows is not an option. Table 1 counts the holes.

Missing weight is heaviest on Dry Van (175 labeled rows). For each of weight, market_index, and quote_signal, the feature builder writes a binary companion column (weight_missing, market_missing, quote_missing) from the raw cell, then fills the numeric column. The flag is 1 when that cell is NaN, or when the column is absent from the file, and 0 when a number was present. Fill-in uses the training-split median for the load's equipment feature, with a global training median as fallback. Those medians are never computed on holdout or validation during model selection. The flag is kept because an imputed equipment median is not the same event as an observed value that happens to equal that median. Since I used gradient-boosted tree models, a tree can split on the indicator without treating the fill-in as a measurement.

Unknown cities map to an unknown code and an unseen-city flag. When coordinates are absent, latitude and longitude fall back to the training city table.

December is a harder challenge because market_index and quote_signal are missing from the file entirely, not merely NaN on some rows. To tackle this, my feature builder also treats both columns as optional, sets the missingness flags to one, and imputes. Requiring those columns

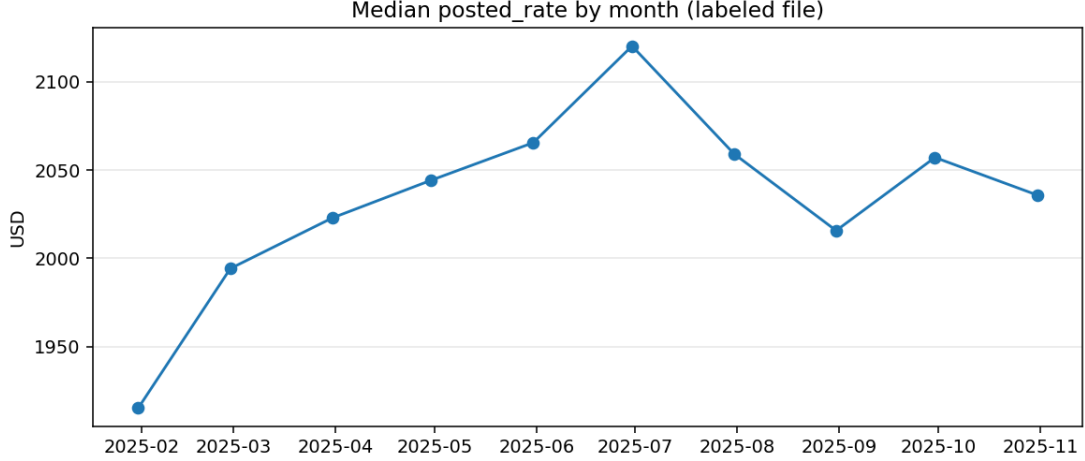


Figure 1: Median posted rate by month in the labeled dataset. September-October is the temporal holdout. November-December are unlabeled.

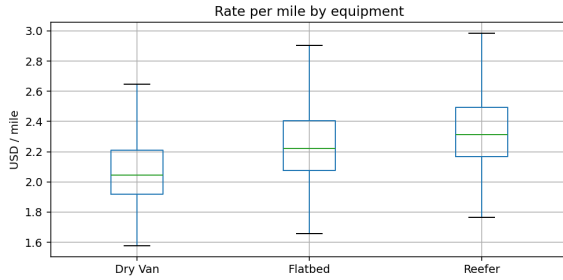


Figure 2: Median posted dollars per mile by equipment type.

would have forced a second pipeline solely to draw a chart.

There are no duplicate `load_id` values, and dates parse cleanly. Distance is clipped at one mile before the logarithm. Predictions are clipped at \$0.01. The submitted validation file ranges from \$222 to \$6,434, all positive, with all 12,000 identifiers from TE-000001 through TE-012000.

6 Feature Representation

One feature builder serves the holdout, the 12,000-row file, and the December series (`src/freight/features.py`). After transform there are 24 columns and no NaNs.

The representation includes raw and log distance; equipment; weight with `weight_missing`; pickup and delivery coordinates; city codes and unseen-city flags; calendar variables; and `market_index` and `quote_signal` with `market_missing` and `quote_missing`. Those three flags are the 0/1 indicators from Section 5, still present after the numeric

columns have been filled. Calendar features are day of week, month, week of year, day of month, weekend, U.S. federal holiday, and days since 1 January 2025. Equipment, pickup, and delivery are categorical inputs to the trees. Coordinates come from the row when present and otherwise from the training city lookup.

Sharing the builder across training and inference keeps the December series and the 12k file on the same column contract.

7 Baselines and Model Selection

Every model is scored on the same September-October holdout. The primary metric is mean absolute error (MAE) in dollars,

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|. \quad (2)$$

I also report MAPE, median absolute error, and dollars-per-mile MAE as diagnostics. But dollar MAE is the most directly interpretable number here, because it tells us how many dollars the prediction is off by, on average.

I start with two simple baselines. One uses median dollars per mile by equipment, multiplied by distance. The other uses Ridge regression on $\log(1 + y)$, with distance, log distance, weight, `market_index`, `quote_signal`, a day index, and equipment dummies. These give us a simple benchmark for seeing whether the tree models are actually better.

Equipment $\times$ miles yields MAE \$229.10. Ridge cuts that to \$121.01, so distance, weight, market,

Model	MAE (\$)	MedAE (\$)	MAPE (%)	RPM MAE
Equipment $\times$ miles	229.10	115.72	10.49	0.238
Ridge on $\log(1 + y)$	121.01	48.28	5.47	0.121
CatBoost	116.51	41.35	5.12	0.113
XGBoost	109.90	34.97	4.73	0.104
LightGBM	108.58	34.79	4.67	0.103

Table 2: September-October holdout. Lower is better for every column.

quote, and a time index already carry signal beyond a constant RPM. Trees improve on Ridge by about twelve dollars MAE (Table 2).

8 Gradient-Boosted Tree Models

LightGBM, XGBoost, and CatBoost are trained on the same split. Trees fit the data because equipment $\times$ distance, city effects, and month are not linear, and because pickup and delivery are high-cardinality categoricals. Forty-eight thousand irregular event rows is a tree problem.

The target is $\log(1 + y)$ with an MAE objective, inverted with `expm1`. Each family has its own `ParameterGrid`. A configuration is scored by September-October MAE, not by k -fold. Early stopping uses that holdout. Row and column subsampling are `subsample` and `colsample_bytree` (CatBoost: `subsample` and `rsm`).

A LightGBM DART configuration was tried. Holdout MAE rose to several thousand dollars, so DART was dropped.

The winner is LightGBM with learning rate 0.03, 63 leaves, minimum 40 samples per child, column subsample 0.9, row subsample 0.7, $\lambda = 1$, and 409 trees. January-August MAE is \$75.56; September-October MAE is \$108.58. A gap of that size is expected on a forward month split.

Feature gain ranks distance and log distance first, then `quote_signal`, pickup, delivery, equipment, weight, market, and the calendar features. That order matches the exploratory analysis.

After the models compete among themselves, the winning configuration is retrained on all 48,000 labeled rows and stored as `models/rate_model.joblib`. That top-performing model artifact is what produces both the 12k file and the December chart, and the one that I deployed to HuggingFace (<https://huggingface.co/Byteroot/lane-rate-lgbm>), with the inference engine, a FastAPI backend, deployed to <https://huggingface.co/spaces/Byteroot/lane-rate>.

9 December Forecasting Behaviour

The December series is a stress test, where structural inputs are frozen. Every row is a 360-mile Dry Van load of 32,000 lb from Lexington to Fort Wayne. The only moving inputs are calendar features. Market and quote are missing on every day.

Figure 3 is the plot written by the supplied `score.py`.

Predicted rates run from \$834.98 to \$843.71 (27 distinct values). That sits a little above the labeled Dry Van median on this lane (\$808, $n = 21$), which is plausible given 21 rows and a different year-end calendar.

The narrow range is the honest output of the feature set, not a modelling failure. Miles, equipment, weight, and the OD pair are constant; the market flags are constant too. Calendar features do almost all of the remaining work. The last three days fall to about \$835. That is a tree split on day-of-month or week, not a demand crash the model claimed to foresee. Christmas is flagged as a holiday and does not produce a spike.

The curve should not be read as a daily market forecast. Without a time-varying market series for the lane, the model has almost nothing from which to infer short-term rate moves.

10 Discussion

Most of the error reduction relative to a naive RPM rule comes from putting distance together with nonlinear effects of equipment, location, shipment traits, and time. Ridge already captures a large share of that, and trees take the rest.

Lane-level models remain a poor fit for this file. The median lane has three rows, and the December series would not move even if those features were added. A more useful next step is a daily market series that actually varies by lane or region. Modelling dollars per mile as an intermediate target, or a two-stage RPM-then-multiply pipeline, is the other experiment worth running. Both should be judged on the same chronological holdout. I stand firm on

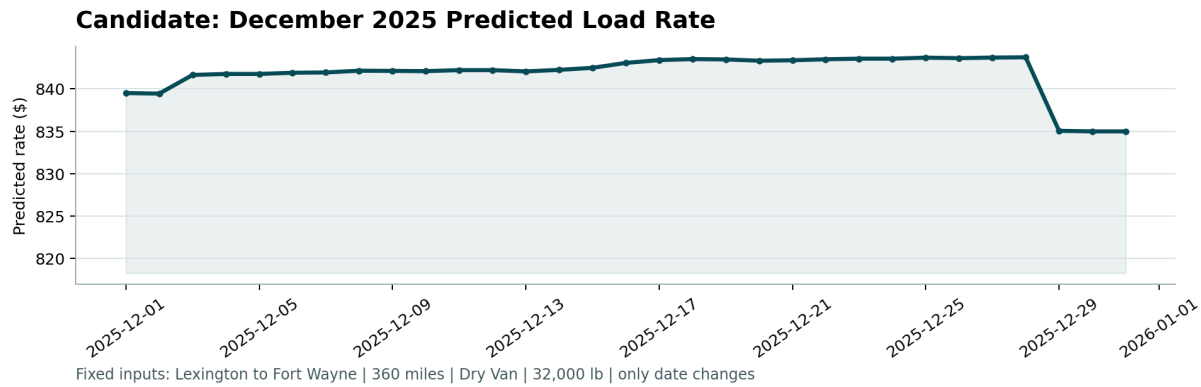


Figure 3: Fixed December prediction chart produced by the provided `score.py`. Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb. Only the date changes.

the belief that simply switching to a random split only to post a lower number would not estimate November-December error more faithfully.

11 Conclusion

A time-aware gradient-boosted tree pipeline predicts posted truck-load rates from tabular shipment data. January-August trains, September-October selects, and the locked LightGBM configuration is refit on all 48,000 labeled rows. Holdout MAE is \$108.58, better than Ridge and the other two tree families. The same model scores 12,000 November-December loads.

Missing features, unseen cities, and sparse lanes were handled in the feature builder, not patched at inference. The December series, which lacks market and quote entirely, shows the limit of calendar features alone. Richer time-varying market information is the most direct path to a more informative short-horizon curve.

Limitations

The labeled window is ten months, so year-over-year seasonality cannot be measured. Many lanes have too few rows for historical lane features. The December series omits `market_index` and `quote_signal`, which caps daily variation. The September-October holdout is a single forward window, additional time-based backtests would give a fuller picture of generalization.

Submission files

This PDF is the written report. It states the train/holdout split (Section 3) and includes the `score.py` December chart (Figure 3). The GitHub repository is <https://github.com/ProsperAsiimwe/spotter>.

github.com/ProsperAsiimwe/spotter. The prediction file is `validation_predictions.csv`: exactly `load_id,predicted_rate` for TE-000001 through TE-012000.

Reproducibility

Python 3.9 or later, from the repository root:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
export PYTHONPATH=src
```

```
python3 scripts/eda.py
python3 scripts/baselines.py
python3 scripts/train.py
python3 scripts/predict.py
```

```
python3 score.py \
  --predictions validation_predictions.csv \
  --december-predictions december-chart-inputs.csv
```

`PYTHONPATH=src` is required so `import freight` works. On macOS, LightGBM needs OpenMP (brew install libomp if the import fails). `score.py` is used without modification from the assignment documents. The December inputs sit in the repository root, not under `data/`.

References

- Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.
- Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu.

2017. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30.

Divyam Madaan, Sumit Chopra, and Kyunghyun Cho. 2025. [Temporal generalization: A reality check](#). *Preprint*, arXiv:2509.23487.

Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. 2018. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31.